COMPUTAÇÃO GRÁFICA · PROJETO AVALIATIVO

Chuva de Meteoros

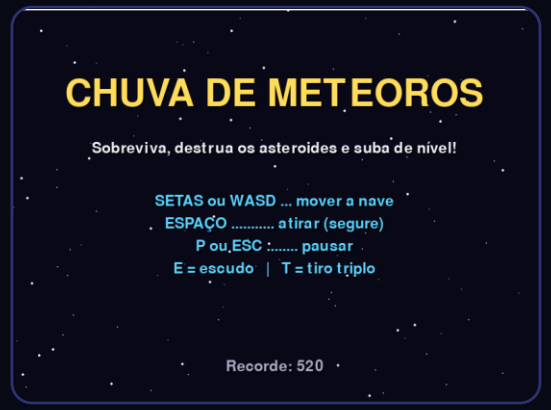
Jogo 2D de nave em Python e Pygame



Gustavo Henrique Alves Costa
Luis Otávio Garcia Santos

Professor Pedro Cândido · Turma [6 Período] ·

Chuva de Meteoros em destaque



1 · Menu com instruções



2 · Jogo, HUD e power-ups



3 · Partículas e colisões



4 · Fim de jogo e reinício

Parallax | Partículas | Animação | Transformações | Procedural | Câmera

Colisão por círculos

Colidem quando a distância² entre os centros é menor que (r1 + r2)².

Progressão

A cada 500 pontos sobe o nível: asteroides mais rápidos e frequentes.

Estados e recorde

Menu, jogo, pausa e fim, com reinício e recorde salvo em arquivo.

Problema de projeto e conceito

Problema

Entregar um jogo 2D completo e estável, com início, desafio, fim e reinício, em escopo pequeno o bastante para ser refinado.

Conceito

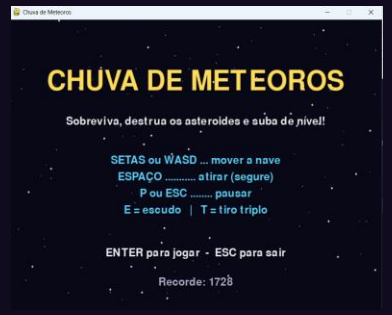
Arcade de nave: desviar e destruir asteroides, coletar power-ups e sobreviver enquanto a dificuldade aumenta.

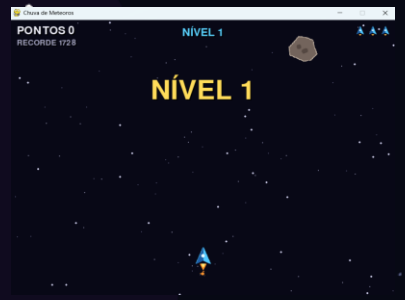
Público e objetivo

Jogadores casuais. Objetivo da partida: fazer o maior número de pontos antes de perder as 3 vidas.

Fluxo da partida

MENU → JOGANDO → PAUSADO → FIM → REINICIAR











Regras, controles e estados

Controles

Setas ou WASD: mover
Espaço: atirar (segurar)
P ou Esc: pausar · Enter: iniciar

Regras

3 vidas; cada batida dá 2 s de invulnerabilidade. E = escudo, T = tiro triplo (8 s). Asteroides grandes se dividem.

Máquina de estados: o estado decide qual tecla faz o quê

```
# resumo de tratar_eventos()
if self.estado == MENU:
    if e.key in (K_RETURN, K_SPACE):
        self.novo_jogo()
elif self.estado == JOGANDO:
    if e.key in (K_p, K_ESCAPE):
        self.estado = PAUSADO
elif self.estado == PAUSADO:
    if e.key in (K_p, K_ESCAPE):
        self.estado = JOGANDO
elif self.estado == FIM:
    if e.key == K_RETURN:
        self.novo_jogo()
```

Cada estado é uma constante (MENU, JOGANDO, PAUSADO, FIM). Isso evita erro de digitação.

Arquitetura do código

Elemento	Responsabilidade
colidem()	Colisão círculo x círculo
CampoEstrelas	Parallax do fundo
Particulas	Explosões e rastro
Tiro, Asteroide, PowerUp	Entidades do jogo
Jogador	Entrada, tiro, power-ups
Jogo	Laço, estados, HUD, recorde

Laço principal: ler, atualizar, desenhar

```
def executar(self):
    while True:
        dt = min(self.relogio.tick(FPS)/1000, 0.05)
        self.tratar_eventos()   # 1) entrada
        self.atualizar(dt)      # 2) estado
        self.desenhar()         # 3) renderiza
```

O papel do dt

tick(FPS) limita a 60 quadros por segundo e devolve os ms do quadro. Com dt em segundos, a velocidade é igual em qualquer PC.

Técnicas de Computação Gráfica

Parallax: 3 camadas, velocidades diferentes

```
# CampoEstrelas.atualizar
for e in self.estrelas:
    vel_y, _, _, fator = self.CAMADAS[e[2]]
    e[1] += vel_y * impulso * dt
    e[0] = (e[0] - vel_x * fator * dt) % LARGURA
    if e[1] > ALTURA:
        e[1] = 0
        e[0] = random.uniform(0, LARGURA)
```

Partículas: vetor girado + fade-out

```
# Particulas.emitir (resumido)
for _ in range(quantidade):
    vel = Vector2(random.uniform(20, vel_max), 0)
    vel = vel.rotate(random.uniform(0, 360))

# Particulas.desenhar
f = vida / total          # 1.0 -> 0.0
cor_f = tuple(int(c * f) for c in cor)
raio = max(1, int(tam * f + 0.5))
```

Animação

3 quadros da chama, 15 trocas por segundo.

Transformações

transform.rotate nos asteroides e na nave; smoothscale no ícone.

Procedural

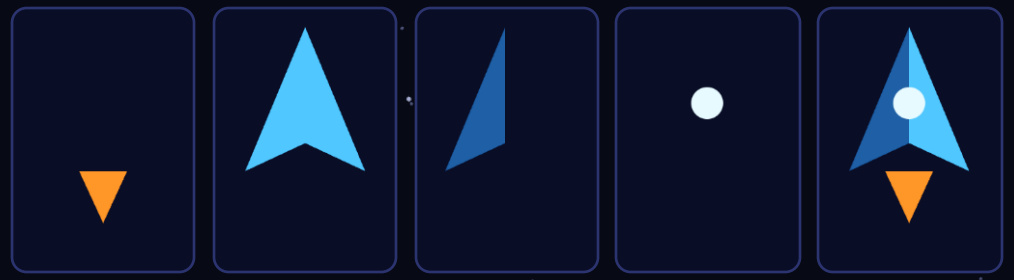
Asteroide = polígono irregular + crateras.

Câmera

Cena desenhada à parte e deslocada (tremor). O HUD fica por cima.

Como o jogo foi desenhado: 0 imagens externas

A nave, peça por peça



1 chama | 2 corpo | 3 sombra | 4 cabine | = nave

O código que monta o sprite

```
# criar_sprite_nave (resumido)
s = pygame.Surface((40, 60), pygame.SRCALPHA)
pygame.draw.polygon(s, laranja, chama)
pygame.draw.polygon(s, azul, corpo)
pygame.draw.circle(s, claro, (20, 21), 4)
return s   # pronto: só copiar com blit
```

Elemento	Como foi desenhado
Nave	polygon + circle, uma vez só
Asteroides	polígono aleatório + rotate
Estrelas	draw.circle em 3 camadas
Partículas	draw.circle com fade-out
Power-ups	círculo pulsante + letra
Textos e HUD	fonte padrão do Pygame

Surface com alpha → **draw.polygon / circle** → **blit: copia na tela** → **flip: mostra o quadro**

Movimento, colisões e progressão

Colisão: distância² < (r1 + r2)²

```
def colidem(pos_a, raio_a, pos_b, raio_b):
    return (pos_a - pos_b).length_squared() \
        < (raio_a + raio_b) ** 2
```

Movimento: vetor normalizado × velocidade × dt

```
direcao = Vector2(dx, dy)
if direcao.length_squared() > 0:
    direcao = direcao.normalize()
self.pos += direcao * self.VELOCIDADE * dt
self.pos.x = max(22, min(LARGURA - 22, self.pos.x))
```

Progressão: sobe o nível a cada 500 pontos

```
novo_nivel = 1 + self.pontos // 500
if novo_nivel > self.nivel:
    self.nivel = novo_nivel
    self.aviso = 2.0       # mostra "NÍVEL N"

# intervalo entre asteroides diminui
espera = 1.2 - 0.1 * (self.nivel - 1)
self.tempo_spawn = max(0.25, espera)
# velocidade: 90..150 + 12 * nivel
```

Evolução do projeto

1 · Concepção

Tema, mecânica e esboços de tela.

2 · Protótipo

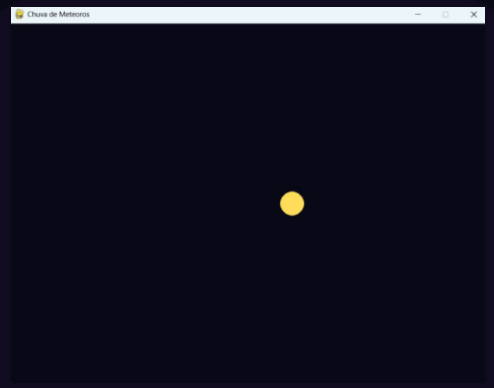
Nave, tiro, laço e colisão.

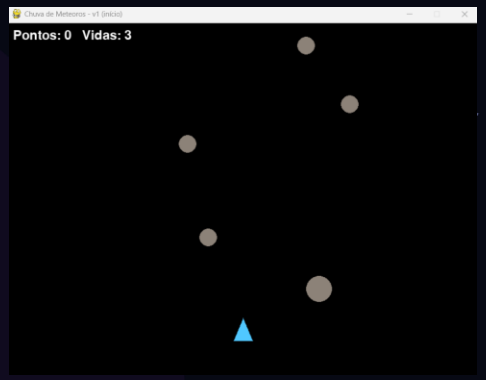
3 · Intermediária

Estados, HUD, progressão.

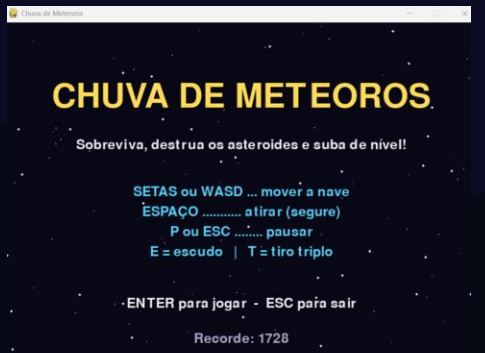
4 · Final

Efeitos, testes e documentação.









Dificuldades, testes e soluções

Diagonal mais rápida

Vetor normalizado: comprimento 1 em qualquer direção.

Velocidade variava com o FPS

Movimento com dt, limitado a 0,05 s.

Colisão injusta

Círculo de colisão menor que o sprite.

Testes

Controles, limites, colisões, pausa, fim, reinício e recorde. [registrar datas]

Trechos que resolveram os problemas

```
# limita dt (evita saltos após travadas)
dt = min(self.relogio.tick(FPS) / 1000, 0.05)

# hitbox menor que o desenho (40 px)
RAIO = 13

# recorde: não quebra sem permissão (resumido)
try:
    ARQ_RECORDE.write_text(json.dumps(dados))
except OSError:
    pass
```

Demonstração

Roteiro da demonstração

1. Menu e instruções
2. Mover, atirar e pegar power-ups
3. Subir de nível
4. Pausar e continuar
5. Perder, ver recorde e reiniciar

Sempre ter à mão

Vídeo curto de segurança e o comando: python main.py



FIM DA APRESENTAÇÃO

Obrigado!

Perguntas?

Chuva de Meteoros · Python e Pygame

Gustavo Henrique Alves Costa
Luis Otavio Garcia Santos

Computação Gráfica · Professor Pedro Cândido

